

Integer and Nonlinear Programming

Robert Hildebrand

Virginia Tech

`rhil@vt.edu`

with contributions from the open-source community

Draft Version — August 1, 2026

Book 2 of a Two-Book Series

Book 1: Introduction to Mathematical Programming

(Linear Programming, Discrete Algorithms, Introduction to Integer Programming)

Book 2: Integer and Nonlinear Programming

(This volume)

Contents

Contents	3
Introduction	1
I Integer Programming	3
1 Integer Programming Formulations	7
1.1 Knapsack Problem	7
1.2 Capital Budgeting	10
1.3 Capacitated Lot Sizing Problem (CLSP)	13
1.4 Facility Location	14
1.4.1 Capacitated Facility Location	14
1.4.2 Uncapacitated Facility Location	17
1.5 Set Covering	19
1.5.1 Covering (Generalizing Set Cover)	23
1.6 Graph Coloring	23
1.7 Basic Modeling Tricks - Using Binary Variables	26
1.7.1 Big M constraints - Activating/Deactivating Inequalities	28
1.7.2 Either Or Constraints	32
1.7.3 If then implications - opposite direction	33
1.7.4 Multi Term Disjunction with application to 2D packing	35
1.8 Generalized k -of- n Disjunction	35
1.8.0.1 Strip Packing Problem	36
1.8.1 SOS1 Constraints	37
1.8.2 SOS2 Constraints	38
1.8.3 Piecewise linear functions with SOS2 constraint	39
1.8.3.1 SOS2 with binary variables	41
1.8.4 Maximizing a minimum	41
1.8.5 Relaxing (nonlinear) equality constraints	42
1.8.6 Connecting to continuous variables	42
1.8.7 Modeling the Exact Absolute Value	42
1.8.7.1 Modeling the Exact 1-Norm	43
1.8.7.2 Modeling the Exact Maximum	43
1.9 Job Shop Scheduling	45
1.9.1 JSSP Components	45
1.9.2 Mathematical Model	47

1.9.3	Job Shop Scheduling Variations	49
1.10	Quadratic Assignment Problem (QAP)	50
1.11	Generalized Assignment Problem (GAP)	51
1.11.1	In special cases	52
1.11.2	Explanation of definition	52
1.12	Other material	52
1.12.1	Binary reformulation of integer variables	52
1.13	Exercises	54
1.14	Literature and Resources	55

Introduction

This book is the second volume in a two-book series on Mathematical Programming and Operations Research.

Prerequisites

This book assumes familiarity with the material covered in **Book 1: Introduction to Mathematical Programming**, including:

- **Linear Programming (LP):** Formulating LP problems, the simplex method, duality theory, and sensitivity analysis
- **Discrete Algorithms:** Graph theory fundamentals, shortest path algorithms, and network flow concepts
- **Introduction to Integer Programming:** Basic IP formulations including knapsack problems, assignment problems, and set covering

What This Book Covers

Part I: Integer Programming provides a comprehensive treatment of integer programming, building on the introductory material from Book 1. Topics include:

- Advanced IP formulations and modeling techniques
- Branch-and-bound and cutting plane algorithms
- Exponential-sized formulations and column generation
- Computational complexity theory (P, NP, NP-complete)
- Heuristics and metaheuristics for combinatorial optimization

Part II: Nonlinear Programming introduces optimization with nonlinear objective functions and constraints:

- Unconstrained optimization: optimality conditions, gradient descent, Newton's method
- One-dimensional optimization methods

- Gradient methods: steepest descent, conjugate gradient
- Constrained optimization: KKT conditions, barrier methods
- Applications to machine learning and data science

How to Use This Book

Each chapter builds upon previous material. Readers should have a solid understanding of linear algebra (matrices, vectors, eigenvalues) and multivariable calculus (gradients, Hessians) for the nonlinear programming sections.

The appendices provide a review of essential linear algebra concepts for reference.

Part I

Integer Programming

[../open-optimization/open-optimization.tex]subfiles

1. Integer Programming Formulations

Outcomes

- A. Learn classic integer programming formulations.
- B. Demonstrate different uses of binary and integer variables.
- C. Demonstrate the format for modeling an optimization problem with sets, parameters, variables, and the model.

In this section, we will describe classical integer programming formulations. These formulations may reflect a real world problem exactly, or may be part of the setup of a real world problem.

1.1 Knapsack Problem

The *knapsack problem* can take different forms depending on if the variables are binary or integer. The binary version means that there is only one item of each item type that can be taken. This is typically illustrated as a backpack (knapsack) and some items to put into it (see Figure 1.1), but has applications in many contexts.

Binary Knapsack Problem:

NP-Complete

Given an non-negative weight vector $\mathbf{a} \in \mathbb{Q}_+^n$, a capacity $b \in \mathbb{Q}_+$, and objective coefficients $\mathbf{c} \in \mathbb{Q}^n$,

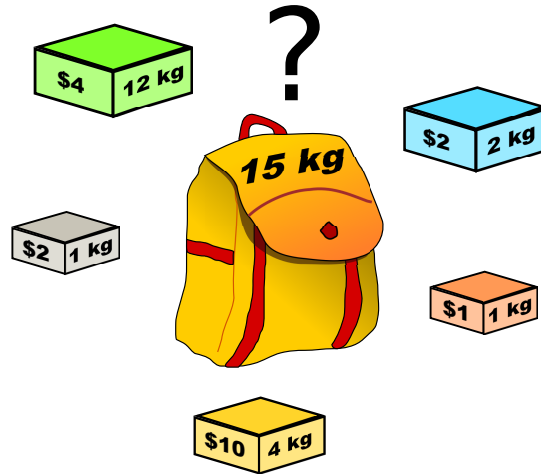
$$\begin{aligned} \max \quad & \mathbf{c}^\top \mathbf{x} \\ \text{s.t.} \quad & \mathbf{a}^\top \mathbf{x} \leq b \\ & \mathbf{x} \in \{0, 1\}^n \end{aligned} \tag{1.1}$$

2

Knapsack Problem: which items should we choose take in the knapsack that maximizes the value while respecting the 15kg weight limit?

¹[wiki/File/knapsack](#), from [wiki/File/knapsack](#). [wiki/File/knapsack](#), [wiki/File/knapsack](#).

²https://en.wikipedia.org/wiki/Knapsack_problem

© wiki/File/knapsack¹

Knapsack Problem: which items should we choose take in the knapsack that maximizes the value while respecting the 15kg weight limit?

Example 1.2: Knapsack

Gurobipy

You have a knapsack (bag) that can only hold $W = 15$ kgs. There are 5 items that you could possibly put into your knapsack. The items (weight, value) are given as: (12 kg, \$4), (2 kg, \$2), (1kg, \$2), (1kg, \$1), (4kg, \$10). Which items should you take to maximize your value in the knapsack? See Figure 1.1.

Variables:

- let $x_i = 0$ if item i is in the bag
- let $x_i = 1$ if item i is not in the bag

Model:

$$\begin{array}{ll}
 \max & 4x_1 + 2x_2 + 2x_3 + 1x_4 + 10x_5 & \text{(Total value)} \\
 \text{s.t.} & 12x_1 + 2x_2 + 1x_3 + 1x_4 + 4x_5 \leq 15 & \text{(Capacity bound)} \\
 & x_i \in \{0, 1\} & \text{for all } i = 1, \dots, 5 \quad \text{(Item taken or not)}
 \end{array}$$

In the integer case, we typically require the variables to be non-negative integers, hence we use the notation $\mathbf{x} \in \mathbb{Z}_+^n$. This setting reflects the fact that instead of single individual items, you have item types of which you can take as many of each type as you like that meets the constraint.

Integer Knapsack Problem:

NP-Complete

Given an non-negative weight vector $\mathbf{a} \in \mathbb{Q}_+^n$, a capacity $b \in \mathbb{Q}_+$, and objective coefficients $c \in \mathbb{Q}^n$,

$$\begin{aligned} \max \quad & \mathbf{c}^\top \mathbf{x} \\ \text{s.t.} \quad & \mathbf{a}^\top \mathbf{x} \leq b \\ & \mathbf{x} \in \mathbb{Z}_+^n \end{aligned} \tag{1.2}$$

We can also consider an equality constrained version

Equality Constrained Integer Knapsack Problem:

NP-Hard

Given an non-negative weight vector $\mathbf{a} \in \mathbb{Q}_+^n$, a capacity $b \in \mathbb{Q}_+$, and objective coefficients $c \in \mathbb{Q}^n$,

$$\max \quad \mathbf{c}^\top \mathbf{x} \tag{1.3}$$

$$\text{s.t.} \quad \mathbf{a}^\top \mathbf{x} = b \tag{1.4}$$

$$\mathbf{x} \in \mathbb{Z}_+^n \tag{1.5}$$

Example 1.5: Min Coins

Gurobipy

Using pennies, nickels, dimes, and quarters, how can you minimize the number of coins you need to to make up a sum of 83¢?

Variables:

- Let p be the number of pennies used
- Let n be the number of nickels used
- Let d be the number of dimes used
- Let q be the number of quarters used

Model:

$$\begin{array}{ll}
 \min & p + n + d + q & \text{(total number of coins used)} \\
 \text{s.t.} & p + 5n + 10d + 25q = 83 & \text{sums to 83¢} \\
 & p, d, n, q \in \mathbb{Z}_+ & \text{(each is a non-negative integer)}
 \end{array}$$

1.2 Capital Budgeting

The *capital budgeting* problem is a nice generalization of the knapsack problem. This problem has the same structure as the knapsack problem, except now it has multiple constraints. We will first describe the problem, give a general model, and then look at an explicit example.

Capital Budgeting:

A firm has n projects it could undertake to maximize revenue, but budget limitations require that not all can be completed.

- Project j expects to produce revenue c_j dollars overall.
- Project j requires investment of a_{ij} dollars in time period i for $i = 1, \dots, m$.
- The capital available to spend in time period i is b_i .

Which projects should the firm invest in to maximize its expected return while satisfying its weekly budget constraints?

We will first provide a general formulation for this problem.

Capital Budgeting Model:

Sets:

- Let $I = \{1, \dots, m\}$ be the set of time periods.
- Let $J = \{1, \dots, n\}$ be the set of possible investments.

Parameters:

- c_j is the expected revenue of investment j for $j \in J$
- b_i is the available capital in time period i for i in I
- a_{ij} is the resources required for investment j in time period i , for i in I , for j in J .

Variables:

- let $x_j = 1$ if investment j is chosen
- let $x_j = 0$ if investment j is not chosen

Model:

$$\begin{aligned}
 \max \quad & \sum_{j=1}^n c_j x_j && \text{(Total Expected Revenue)} \\
 \text{s.t.} \quad & \sum_{j=1}^n a_{ij} x_j \leq b_i, \quad i = 1, \dots, m && \text{(Resource constraint week } i) \\
 & x_j \in \{0, 1\}, \quad j = 1, \dots, n
 \end{aligned}$$

Consider the example given in the following table.

Project	$\mathbb{E}[\text{Revenue}]$	Resources required in week 1	Resources required in week 2
1	10	3	4
2	8	1	2
3	6	2	1
Resources available		5	6

Given this data, we can set up our problem explicitly as follows

Example 1.8: Capital Budgeting

Excel PuLP Gurobipy

Sets:

- Let $I = \{1, 2\}$ be the set of time periods.
- Let $J = \{1, 2, 3\}$ be the set of possible investments.

Parameters:

- c_j is given in column “ $\mathbb{E}[\text{Revenue}]$ ”.
- b_i is given in row “Resources available”.
- a_{ij} is given in row j , and column for week i .

Variables:

- let $x_j = 1$ if investment j is chosen
- let $x_j = 0$ if investment j is not chosen

The explicit model is given by

Model:

$$\begin{aligned}
 \max \quad & 10x_1 + 8x_2 + 6x_3 && \text{(Total Expected Revenue)} \\
 \text{s.t.} \quad & 3x_1 + 1x_2 + 2x_3 \leq 5 && \text{(Resource constraint week 1)} \\
 & 4x_1 + 2x_2 + 1x_3 \leq 6 && \text{(Resource constraint week 2)} \\
 & x_j \in \{0, 1\}, \quad j = 1, 2, 3
 \end{aligned}$$

1.3 Capacitated Lot Sizing Problem (CLSP)

The *Capacitated Lot Sizing Problem (CLSP)* is a fundamental optimization problem in production planning, where the goal is to determine an optimal production schedule that minimizes total costs while adhering to capacity constraints. The objective function accounts for production costs, setup costs, and inventory holding costs over a given planning horizon.

Let T denote the number of discrete time periods, indexed by $t \in \{1, 2, \dots, T\}$. The problem parameters are defined as follows:

- d_t is the demand in period t ,
- p_t represents the unit production cost,
- f_t is the fixed setup cost incurred if production takes place,
- h_t denotes the unit inventory holding cost, and
- c_t represents the maximum production capacity in period t .

The decision variables are:

- x_t denotes the quantity produced in period t ,
- s_t is the inventory level at the end of period t , and
- y_t is a binary variable that takes value 1 if production occurs in period t , and 0 otherwise.

The CLSP can be formulated as the following mixed-integer linear program:

$$\begin{aligned}
 \min \quad & \sum_{t=1}^T (p_t x_t + f_t y_t + h_t s_t) && \text{(Minimize total cost)} \\
 \text{s.t.} \quad & s_{t-1} + x_t - d_t = s_t && \text{for all } t = 1, 2, \dots, T \quad \text{(Inventory balance)} \\
 & x_t \leq y_t c_t && \text{for all } t = 1, 2, \dots, T \quad \text{(Capacity and setup constraint)} \\
 & x_t, s_t \geq 0 && \text{for all } t = 1, 2, \dots, T \quad \text{(Non-negativity constraints)} \\
 & y_t \in \{0, 1\} && \text{for all } t = 1, 2, \dots, T \quad \text{(Binary production decision)}
 \end{aligned}$$

The objective function (**Minimize total cost**) minimizes the total cost over the planning horizon, incorporating production, setup, and inventory holding costs. The inventory balance constraint (**Inventory balance**) ensures that demand in each period is met using either inventory from the previous period or newly produced units. The constraint (**Capacity and setup constraint**) enforces production capacity limits and ensures that a setup cost is incurred when production occurs. Constraints (**Non-negativity constraints**) enforce non-negativity on production and inventory levels, while (**Binary production decision**) ensures that the setup decision is binary.

For simplicity, we assume an initial inventory level of zero.

1.4 Facility Location

The basic model of the facility location problem is to determine where to place your stores or facilities in order to be close to all of your customers and hence reduce the costs transportation to your customers. Each customer is known to have a certain demand for a product, and each facility has a capacity on how much of that demand it can satisfy. Furthermore, we need to consider the cost of building the facility in a given location.

This basic framework can be applied in many types of problems and there are a number of variants to this problem. We will address two variants: the *capacitated facility location problem* and the *uncapacitated facility location problem*.

1.4.1. Capacitated Facility Location

We present 3 ways to model this problem.

Capacitated Facility Location 1:

NP-Complete

Given connection costs c_{ij} , fixed building costs f_i , demands d_j , and facility capacities u_i , the **capacitated facility location** problem is formulated as follows:

Sets:

- Let $I = \{1, \dots, n\}$ be the set of facilities.
- Let $J = \{1, \dots, m\}$ be the set of customers.

Parameters:

- f_i — the cost of opening facility i .
- c_{ij} — the cost of serving one unit of demand of customer j from facility i .
- u_i — the capacity (in units) of facility i .
- d_j — the total demand (in units) of customer j .

Variables:

- $y_i \in \{0, 1\}$ — equals 1 if facility i is opened; 0 otherwise.
- $x_{ij} \geq 0$ — the number of units of customer j 's demand served by facility i .

Model:

$$\begin{aligned}
\min \quad & \sum_{i=1}^n \sum_{j=1}^m c_{ij} x_{ij} + \sum_{i=1}^n f_i y_i && \text{(Total cost)} \\
\text{s.t.} \quad & \sum_{i=1}^n x_{ij} = d_j && \text{for all } j = 1, \dots, m \quad \text{(Meet demand)} \\
& \sum_{j=1}^m x_{ij} \leq u_i y_i && \text{for all } i = 1, \dots, n \quad \text{(Facility capacity)} \\
& x_{ij} \geq 0 && \text{for all } i = 1, \dots, n, j = 1, \dots, m \quad \text{(Nonnegativity)} \\
& y_i \in \{0, 1\} && \text{for all } i = 1, \dots, n \quad \text{(Open facility)}
\end{aligned}$$

Next we present a method that models it tracking the fraction of demand served.

Capacitated Facility Location 2:*NP-Complete*

Given costs connections c_{ij} and fixed building costs f_i , demands d_j and capacities u_i , the capacitated facility location problem is

Sets:

- Let $I = \{1, \dots, n\}$ be the set of facilities.
- Let $J = \{1, \dots, m\}$ be the set of customers.

Parameters:

- f_i - the cost of opening facility i .
- c_{ij} - the cost of fulfilling the complete demand of customer j from facility i .
- u_i - the capacity of facility i .
- d_j - the demand by customer j .
- c_j is given in column "E[Revenue]".
- b_i is given in row "Resources available".
- a_{ij} given in row j , and column for week i .

Variables:

- Let

$$y_i = \begin{cases} 1 & \text{if we open facility } i, \\ 0 & \text{otherwise.} \end{cases}$$

- Let $x_{ij} \geq 0$ be the fraction of demand of customer j satisfied by facility i .
- let $x_i = 1$ if investment i is not chosen

Model:

$$\begin{aligned}
\min \quad & \sum_{i=1}^n \sum_{j=1}^m c_{ij} x_{ij} + \sum_{i=1}^n f_i y_i && \text{(total cost)} \\
\text{s.t.} \quad & \sum_{i=1}^n x_{ij} = 1 && \text{for all } j = 1, \dots, m \quad \text{(assign demand to facility)} \\
& \sum_{j=1}^m d_j x_{ij} \leq u_i y_i && \text{for all } i = 1, \dots, n \quad \text{(capacity of facility } i\text{)} \\
& x_{ij} \geq 0 && \text{for all } i = 1, \dots, n \text{ and } j = 1, \dots, m \\
& && \text{(nonnegative fraction of demand satisfied)} \\
& y_i \in \{0, 1\} && \text{for all } i = 1, \dots, n \quad \text{(open/not open facility)}
\end{aligned}$$

Finally, we present a version that has the same variables, but the constraints model the feasible solutions in a slightly different way.

$$\begin{aligned}
\min \quad & \sum_{i=1}^m \sum_{j=1}^n c_{ij} x_{ij} + \sum_{i=1}^m f_i y_i \\
\text{s.t.} \quad & \sum_{i=1}^m x_{ij} = 1, \quad j = 1, \dots, n \\
& \sum_{j=1}^n d_j x_{ij} \leq u_i && \text{for all } i = 1, \dots, m \quad \text{(capacity of facility } i\text{)} \\
& x_{ij} \leq y_i, \quad i = 1, \dots, m, \quad j = 1, \dots, n \\
& x_{ij} \geq 0 \quad i = 1, \dots, m, \quad j = 1, \dots, n \\
& y_i \in \{0, 1\}, \quad i = 1, \dots, m
\end{aligned}$$

Example 1.11: Capacitated Facility Location: Retail Distribution Example

GurobiPy

Context: A retail company plans to establish distribution centers across a country to serve its stores efficiently. The company faces decisions on which distribution centers to open and which stores each center should serve. Costs associated with opening each center and serving stores from them are known, and each center has a maximum capacity. Additionally, each store has a known demand.

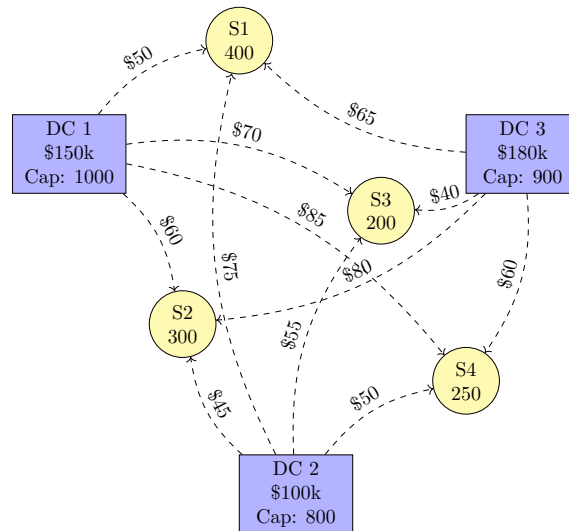
Given Data: (distribution-data.xlsx)

- Number of potential distribution centers (n): 3
- Number of stores (m): 4

Combined Data for Distribution Centers and Stores

Center	f_i	u_i	c_{ij} (Cost to Serve a Store)			
	(\$)	(units)	Store 1	Store 2	Store 3	Store 4
Center 1	150,000	100	\$50	\$60	\$70	\$85
Center 2	100,000	80	\$75	\$45	\$55	\$50
Center 3	180,000	90	\$65	\$80	\$40	\$60
Store Demand d_j			400	300	200	250

Model Formulation: The capacitated facility location model described earlier can be applied to this scenario with the given data to determine the optimal number and location of distribution centers to open, and which stores each center should serve.



1.4.2. Uncapacitated Facility Location

Uncapacitated Facility Location:

NP-Complete

Given costs connections c_{ij} and fixed building costs f_i , the uncapacitated facility location problem is

$$\begin{aligned}
 \min \quad & \sum_{i=1}^n \sum_{j=1}^m c_{ij} z_{ij} + \sum_{i=1}^n f_i x_i \\
 \text{s.t.} \quad & \sum_{i=1}^n z_{ij} = 1 \text{ for all } j = 1, \dots, m \\
 & \sum_{j=1}^m z_{ij} \leq M x_i \text{ for all } i = 1, \dots, n \\
 & z_{ij} \in \{0, 1\} \text{ for all } i = 1, \dots, n \text{ and } j = 1, \dots, m \\
 & x_i \in \{0, 1\} \text{ for all } i = 1, \dots, n
 \end{aligned} \tag{1.1}$$

Here M is a large number and can be chosen as $M = m$, but could be refined smaller if more context is known.

Alternative model!

$$\begin{aligned}
 \min \quad & \sum_{i=1}^n \sum_{j=1}^m c_{ij} z_{ij} + \sum_{i=1}^n f_i x_i \\
 \text{s.t.} \quad & \sum_{i=1}^n z_{ij} = 1 \text{ for all } j = 1, \dots, m \\
 & z_{ij} \leq x_i \text{ for all } i = 1, \dots, n \text{ for all } j = 1, \dots, m \\
 & z_{ij} \in \{0, 1\} \text{ for all } i = 1, \dots, n \text{ and } j = 1, \dots, m \\
 & x_i \in \{0, 1\} \text{ for all } i = 1, \dots, n
 \end{aligned} \tag{1.2}$$

1.5 Set Covering

The *set covering* problem can be used for a wide array of problems. We will see several examples in this section.

Set Covering:

NP-Complete

Given a set V with subsets $V_1, \dots, V_l$, determine the smallest subset $S \subseteq V$ such that $S \cap V_i \neq \emptyset$ for all $i = 1, \dots, l$.

The set cover problem can be modeled as

$$\begin{aligned} \min \quad & \mathbf{1}^\top x \\ \text{s.t.} \quad & \sum_{v \in V_i} x_v \geq 1 \text{ for all } i = 1, \dots, l \\ & x_v \in \{0, 1\} \text{ for all } v \in V \end{aligned} \tag{1.1}$$

where x_v is a 0/1 variable that takes the value 1 if we include item j in set S and 0 if we do not include it in the set S .

Example 1.14: Capital Budgeting <https://github.com/open-optimization/open-optimization-or-examples/blob/master/integer-programming/capital-budgeting.ipynb>

Sets:

- Let $I = \{1, 2\}$ be the set of time periods.
- Let $J = \{1, 2, 3\}$ be the set of possible investments.

Parameters:

- c_j is given in column " $\mathbb{E}[\text{Revenue}]$ ".
- b_i is given in row "Resources available".
- a_{ij} given in row j , and column for week i .

Variables:

- let $x_i = 0$ if investment i is chosen
- let $x_i = 1$ if investment i is not chosen

Model:

$$\max \quad 10x_1 + 8x_2 + 6x_3 \quad \text{(Total Expected Revenue)}$$

One specific type of set cover problem is the *vertex cover* problem.

Example: Vertex Cover:*NP-Complete*

Given a graph $G = (V, E)$ of vertices and edges, we want to find a smallest size subset $S \subseteq V$ such that every for every $e = (v, u) \in E$, either u or v is in S .

We can write this as a mathematical program in the form:

$$\begin{aligned} \min \quad & \mathbf{1}^\top x \\ \text{s.t.} \quad & x_u + x_v \geq 1 \text{ for all } (u, v) \in E \\ & x_v \in \{0, 1\} \text{ for all } v \in V. \end{aligned} \quad (1.2)$$

Example 1.16: Set cover: Fire station placement

<https://github.com/open-optimization/open-optimization-or-examples/blob/master/integer-programming/fire-station-covering.ipynb>

In the fire station problem, we seek to choose locations for fire stations such that any district either contains a fire station, or neighbors a district that contains a fire station. Figure 1.3 depicts the set of districts and an example placement of locations of fire stations. How can we minimize the the total number of fire stations that we need?

Sets:

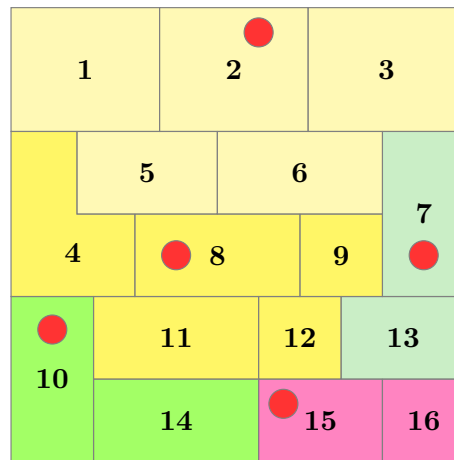
- Let V be the set of districts ($V = \{1, \dots, 16\}$)
- Let V_i be the set of districts that neighbor district i (e.g. $V_1 = \{2, 4, 5\}$).

Variables:

- let $x_i = 1$ if district i is chosen to have a fire station.
- let $x_i = 0$ otherwise.

Model:

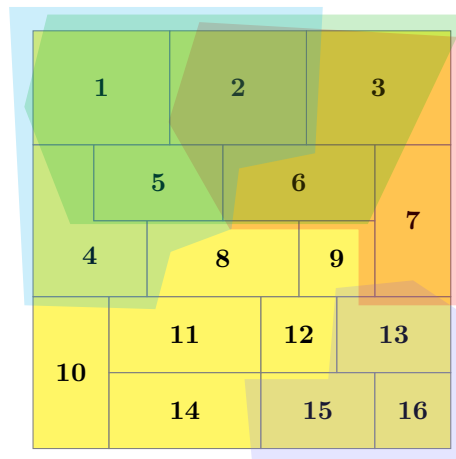
$$\begin{aligned}
\min \quad & \sum_{i \in V} x_i && (\# \text{ open fire stations}) \\
\text{s.t.} \quad & x_i + \sum_{j \in V_i} x_j \geq 1 && \text{for all } i \in V \quad (\text{Station proximity requirement}) \\
& x_i \in \{0, 1\} && \text{for all } i \in V \quad (\text{station either open or closed})
\end{aligned}$$

© tikz/Illustration1.pdf³**Layout of districts and possible locations of fire stations.****Set Covering - Matrix description:***NP-Complete*

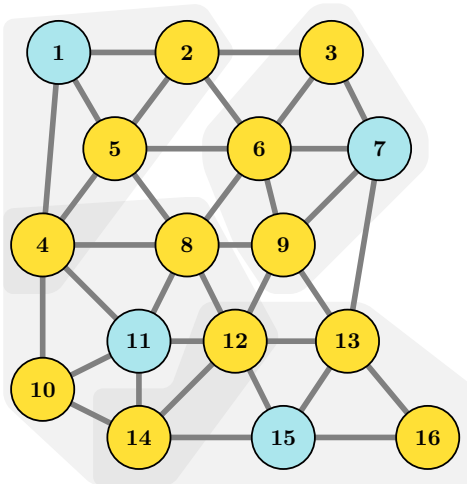
Given a non-negative matrix $A \in \{0, 1\}^{m \times n}$, a non-negative vector, and an objective vector $c \in \mathbb{R}^n$, the set cover problem is

$$\begin{aligned}
\min \quad & c^\top x \\
\text{s.t.} \quad & Ax \geq 1 \\
& x \in \{0, 1\}^n.
\end{aligned} \tag{1.3}$$

³tikz/Illustration1.pdf, from tikz/Illustration1.pdf. tikz/Illustration1.pdf, tikz/Illustration1.pdf.⁴tikz/Illustration2.pdf, from tikz/Illustration2.pdf. tikz/Illustration2.pdf, tikz/Illustration2.pdf.⁵tikz/Illustration3.pdf, from tikz/Illustration3.pdf. tikz/Illustration3.pdf, tikz/Illustration3.pdf.

© tikz/Illustration2.pdf⁴

Set cover representation of fire station problem. For example, choosing district 16 to have a fire station covers districts 13, 15, and 16.

© tikz/Illustration3.pdf⁵

Graph representation of fire station problem. Every node is connected to a chosen node by an edge

Example 1.18: Vertex Cover with matrix

An alternate way to solve [Example: Vertex Cover](#) is to define the *incidence matrix* A of the graph. The incidence matrix is a $|E| \times |V|$ matrix with $\{0, 1\}$ entries. Each row corresponds to an edge e and each column corresponds to a node v . For an edge $e = (u, v)$, the corresponding row has a 1 in columns corresponding to the nodes u and v , and a 0 everywhere else. Hence, there are exactly two 1's per row. Applying the formulation above in [Set Covering - Matrix](#) description models the problem.

1.5.1. Covering (Generalizing Set Cover)

We could also allow for a more general type of set covering where we have non-negative integer variables and a right hand side that has values other than 1.

Covering:

NP-Complete

Given a non-negative matrix $A \in \mathbb{Z}_+^{m \times n}$, a non-negative vector $b \in \mathbb{Z}^m$, and an objective vector $c \in \mathbb{R}^n$, the set cover problem is

$$\begin{aligned} \max \quad & c^\top x \\ \text{s.t.} \quad & Ax \geq b \\ & x \in \mathbb{Z}_+^n. \end{aligned} \tag{1.4}$$

Example 1.20: Nurse Scheduling

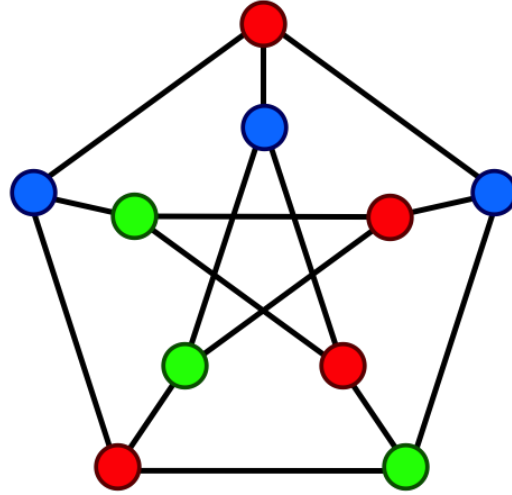
code:nurse-scheduling

1.6 Graph Coloring

Graph coloring Figure 1.6 is is problem of finding a minimum coloring in a graph can be formulated in many ways. Each vertex is assigned a color and two vertices cannot share the same color if they are connected by an edge.

In order to present integer programming formulations, we use x_{ij} to denote binary variables, with $i \in V$ and $1 \leq j \leq n$, where $x_{ij} = 1$ if color j is assigned to vertex i and $x_{ij} = 0$ otherwise. We also define n binary variables w_j for $j = 1, \dots, n$, that indicate whether color j is used in some node, i.e., $w_j = 1$ if $x_{ij} = 1$ for

⁶wiki/File/Petersen-graph-3-coloring.png, from wiki/File/Petersen-graph-3-coloring.png.
wiki/File/Petersen-graph-3-coloring.png, wiki/File/Petersen-graph-3-coloring.png.



© wiki/File/Petersen-graph-3-coloring.png⁶
 wiki/File/Petersen-graph-3-coloring.png

some vertex i . The following is a classical integer programming formulation:

$$\begin{aligned}
 \min \quad & \sum_{j=1}^n w_j \\
 \sum_{j=1}^n x_{ij} &= 1 \quad \forall i \in V, \\
 x_{ij} + x_{kj} &\leq w_j \quad \forall \{i, k\} \in E, \quad 1 \leq j \leq n, \\
 x_{ij} &\in \{0, 1\} \quad \forall i \in V, 1 \leq j \leq n \\
 w_j &\in \{0, 1\} \quad 1 \leq j \leq n.
 \end{aligned}$$

We can improve upon this model. We suggest two possible models from [7]. Please read their paper for more advanced models with additional families of cuts added.

Color order model: Given a (k) -coloring, selecting any k colors from the set $\{1, \dots, n\}$ results in a valid solution, and these solutions are deemed equivalent. To reduce redundancy among these solutions, we stipulate that color j can only be allocated to a vertex if color $j - 1$ has previously been assigned. By doing so, we exclude symmetrical (k) -colorings that use colors with labels exceeding k . To define these valid solutions, we simply need to incorporate the subsequent set of constraints:

$$\begin{aligned}
 w_j &\leq \sum_{i \in V} x_{ij} \quad \forall 1 \leq j \leq n, \\
 w_j &\geq w_{j+1} \quad \forall 1 \leq j \leq n - 1.
 \end{aligned}$$

The reduction of feasible solutions is very significant.

Independent set order model: In a (k) -coloring, permutations of the initial k colors lead to equivalent outcomes. To reduce the occurrence of these similar solutions, the subsequent model introduces stricter constraints than the previous one. It mandates that the count of vertices colored with j should be at least

equal to or surpass the count of vertices colored with $j + 1$. We incorporate the ensuing inequalities:

$$w_j \leq \sum_{i \in V} x_{ij} \quad \forall 1 \leq j \leq n$$

$$\sum_{i=1}^n x_{ij} \geq \sum_{i=1}^n x_{i,j+1} \quad \forall 1 \leq j \leq n-1.$$

As an exercise, try implementing these different models and compare the solve times.

1.7 Basic Modeling Tricks - Using Binary Variables

In this section, we describe ways to model a variety of constraints that commonly appear in practice. The goal is changing constraints described in words to constraints defined by math.

Binary variables can allow you to model many types of constraints. We discuss here various logical constraints where we assume that $x_i \in \{0, 1\}$ for $i = 1, \dots, n$. We will take the meaning of the variable to be selecting an item.

1. If item i is selected, then item j is also selected.

$$x_i \leq x_j \quad (1.1)$$

- (a) If any of items $1, \dots, 5$ are selected, then item 6 is selected.

$$x_1 + x_2 + \dots + x_5 \leq 5 \cdot x_6 \quad (1.2)$$

Alternatively!

$$x_i \leq x_6 \quad \text{for all } i = 1, \dots, 5 \quad (1.3)$$

2. If item j is not selected, then item i is not selected.

$$x_i \leq x_j \quad (1.4)$$

- (a) If item j is not selected, then all items $1, \dots, i$ are not selected.

$$x_1 + x_2 + \dots + x_i \leq i \cdot x_j \quad (1.5)$$

3. If item j is not selected, then item i is not selected.

$$x_i \leq x_j \quad (1.6)$$

4. Either item i is selected or item j is selected, but not both.

$$x_i + x_j = 1 \quad (1.7)$$

5. Item i is selected or item j is selected or both.

$$x_i + x_j \geq 1 \quad (1.8)$$

6. If item i is selected, then item j is not selected.

$$x_j \leq (1 - x_i) \quad (1.9)$$

7. At most one of items i, j , and k are selected.

$$x_i + x_j + x_k \leq 1 \quad (1.10)$$

8. At most two of items i, j , and k are selected.

$$x_i + x_j + x_k \leq 2 \quad (1.11)$$

9. Exactly one of items i, j , and k are selected.

$$x_i + x_j + x_k = 1 \quad (1.12)$$

These tricks can be connected to create different function values.

Example 1.1. Variable takes one of three values Suppose that the variable x should take one of the three values $\{4, 8, 13\}$. This can be modeled using three binary variables as

$$\begin{aligned} x &= 4z_1 + 8z_2 + 13z_3 \\ z_1 + z_2 + z_3 &= 1 \\ z_i &\in \{0, 1\} \text{ for } i = 1, 2, 3. \end{aligned}$$

As a convenient addition, if we want to add the possibility that it takes the value 0, then we can model this as

$$\begin{aligned} x &= 4z_1 + 8z_2 + 13z_3 \\ z_1 + z_2 + z_3 &\leq 1 \\ z_i &\in \{0, 1\} \text{ for } i = 1, 2, 3. \end{aligned}$$

We can also model variable increases at different amounts.

Example 1.2. Discount for buying more Suppose you can choose to buy 1, 2, or 3 units of a product, each with a decreasing cost. The first unit is \$10, the second is \$5, and the third unit is \$3.

$$\begin{aligned} x &= 10z_1 + 5z_2 + 3z_3 \\ z_1 &\geq z_2 \geq z_3 \\ z_i &\in \{0, 1\} \text{ for } i = 1, 2, 3. \end{aligned}$$

Here, z_i represents if we buy the i th unit. The inequality constraints impose that if we buy unit j , then we must buy all units i with $i < j$.

In this section, we describe ways to model a variety of constraints that commonly appear in practice. The goal is changing constraints described in words to constraints defined by math.

1.7.1. Big M constraints - Activating/Deactivating Inequalities

Big M comes again! It's extremely useful when trying to activate constraints based on a binary variable.

For instance, if we don't rent a bus, then we can have at most 3 passengers join us on our trip. Consider passengers A, B, C, D, E and let $x_i \in \{0, 1\}$ be 1 if we take passenger i and 0 otherwise. We can model the constraint that we can have at most 5 passengers as

$$x_A + x_B + x_C + x_D + x_E \leq 3.$$

We want to be able to activate this constraint in the event that we don't rent a bus.

Let $\delta \in \{0, 1\}$ be 1 if rent a bus, and 0 otherwise.

Then we want to say

If $\delta = 0$, then

$$x_A + x_B + x_C + x_D + x_E \leq 3.$$

We can formulate this using a big-M constraint as

$$x_A + x_B + x_C + x_D + x_E \leq 3 + M\delta. \quad (1.13)$$

Notice the two case

$$\begin{cases} x_A + x_B + x_C + x_D + x_E \leq 3 & \text{if } \delta = 0 \\ x_A + x_B + x_C + x_D + x_E \leq 3 + M & \text{if } \delta = 1 \end{cases}$$

In the second case, we choose M to be so large, that the second case inequality is vacuous. That said, choosing smaller M values (that are still valid) will help the computer program solve the problem faster. In this case, it suffices to let $M = 2$.

We can speak about this technique more generally as

Big-M: If then:

We aim to model the relationship

$$\text{If } \delta = 0, \text{ then } a^\top x \leq b. \quad (1.14)$$

By letting M be an upper bound on the quantity $a^\top x - b$, we can model this condition as

$$\begin{aligned} a^\top x - b &\leq M\delta \\ \delta &\in \{0, 1\} \end{aligned} \quad (1.15)$$

Below are several applications of this modeling technique:

- **Facility Location with Fixed Costs:**

To ensure that a facility i must be open in order to serve customer j , we write

$$x_{ij} \leq M\delta_i,$$

where x_{ij} is the quantity shipped and $\delta_i = 1$ if facility i is open.

- **Piecewise Linear or Step Costs:**

- **Power Plant Operation Constraints:**

A generator can only produce power when it is turned on. Letting $\delta_t = 1$ indicate that the generator is on at time t , we write:

$$P_{\min}\delta_t \leq P_t \leq P_{\max}\delta_t.$$

- **Task Scheduling with Precedence Constraints:**

To ensure task B starts only after task A ends, if a binary $\delta = 1$ indicates the precedence relation is active, we write:

$$\text{Start}_B \geq \text{End}_A - M(1 - \delta).$$

- **Routing and Time Windows:**

In vehicle routing problems, to model time consistency across arcs, we use:

$$t_j \geq t_i + s_i - M(1 - \delta_{ij}),$$

where $\delta_{ij} = 1$ indicates travel from node i to node j , and s_i is service time at node i .

- **Production Planning and Setup Costs:**

To ensure production x_t is positive only if a setup decision $\delta_t = 1$ is made:

$$x_t \leq M\delta_t.$$

Example 1.3. Network Flow with Toll Charges Using Big-M Constraints Consider a transportation network where some arcs incur a toll cost if used. Let:

- $G = (V, A)$ be a directed graph with nodes V and arcs A .
- $x_{ij} \geq 0$ be the flow on arc $(i, j) \in A$.
- $\delta_{ij} \in \{0, 1\}$ indicate whether arc (i, j) is used.
- c_{ij} be the per-unit flow cost on arc (i, j) .
- t_{ij} be the fixed toll incurred if arc (i, j) is used.
- M_{ij} be a large constant upper bound on possible flow through arc (i, j) .

The objective is to route flow from a source node s to a sink node t at minimum total cost, including tolls when applicable:

$$\min \sum_{(i,j) \in A} (c_{ij}x_{ij} + t_{ij}\delta_{ij})$$

Subject to:

$$\sum_{j:(i,j) \in A} x_{ij} - \sum_{j:(j,i) \in A} x_{ji} = \begin{cases} 1 & \text{if } i = s \\ -1 & \text{if } i = t \\ 0 & \text{otherwise} \end{cases} \quad (\text{Flow conservation})$$

$$\begin{aligned} x_{ij} &\leq M_{ij}\delta_{ij} && (\text{Big-M: toll applies only if arc is used}) \\ x_{ij} &\geq 0, \quad \delta_{ij} \in \{0, 1\} && \text{for all } (i, j) \in A \end{aligned}$$

NYC Transportation Network with Tolls

Figure 1.7 illustrates a simplified transportation network over the five boroughs of New York City, with selected connections to model bridges and tunnels. Each node represents a borough, and directed arcs represent feasible travel routes. Some of these arcs include tolls, which are incurred only in one direction, mimicking real-world tolling policies.

NODES. The nodes in the graph correspond to the following boroughs:

- Manhattan
- Brooklyn
- Queens
- Bronx
- Staten Island

ARCS. The directed arcs model travel between boroughs, with an optional toll charge. These arcs represent crossings such as bridges or tunnels, and are defined as follows:

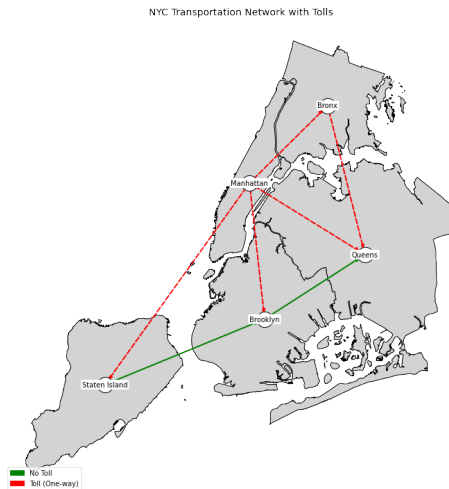
From	To	Toll
Manhattan	Brooklyn	Yes (e.g., Brooklyn-Battery Tunnel)
Manhattan	Queens	Yes (e.g., Queens-Midtown Tunnel)
Brooklyn	Queens	No
Manhattan	Bronx	Yes (e.g., RFK/Triborough Bridge)
Bronx	Queens	Yes
Manhattan	Staten Island	Yes (e.g., Verrazzano-Narrows Bridge)
Staten Island	Brooklyn	No

TOLL MODELING. Each arc (i, j) in the network has an associated binary toll flag. If a toll applies, the arc is subject to a fixed toll cost only if it is used. This can be modeled in a mixed-integer program using a binary variable $\delta_{ij} \in \{0, 1\}$ that indicates whether the arc is selected. A Big- M constraint of the form:

$$x_{ij} \leq M \cdot \delta_{ij}$$

can be used to enforce toll cost and flow constraints, where x_{ij} is the flow over arc (i, j) , and M is a suitably large upper bound on the possible flow.

LEGEND. In the figure, solid green arrows represent toll-free travel, while dashed red arrows indicate that a toll is applied in the direction of the arc.



1.7.2. Either Or Constraints

“At least one of these constraints holds” is what we would like to model. Equivalently, we can phrase this as an *inclusive or* constraint. This can be modeled with a pair of Big-M constraints.

Either Or:

$$\text{Either } a^\top x \leq b \text{ or } c^\top x \leq d \text{ holds} \quad (1.16)$$

can be modeled as

$$\begin{aligned} a^\top x - b &\leq M_1 \delta \\ c^\top x - d &\leq M_2(1 - \delta) \\ \delta &\in \{0, 1\}, \end{aligned} \quad (1.17)$$

where M_1 is an upper bound on $a^\top x - b$ and M_2 is an upper bound on $c^\top x - d$.

INTERPRETATION.

- If $\delta = 0$, then the first constraint becomes $a^\top x \leq b$, and the second becomes $c^\top x \leq d + M_2$, which is trivially satisfied if M_2 is large enough.
- If $\delta = 1$, then the second constraint becomes $c^\top x \leq d$, while the first is relaxed as $a^\top x \leq b + M_1$.

CHOOSING BIG-M VALUES. The constants M_1 and M_2 should be chosen as valid upper bounds on the expressions $a^\top x - b$ and $c^\top x - d$, respectively, over the feasible region. Overly large values may weaken the LP relaxation and cause numerical instability, so careful bounding is recommended whenever possible.

USE CASES. Either-or constraints naturally arise in problems with conditional decisions, such as:

- If a machine is used, then either a certain energy constraint or a safety constraint must be met.
- In facility planning, either a site must be built with one configuration or another.
- In piecewise models where a variable must fall into one of two feasible regions.

Example 1.4. Either 2 buses or 10 cars are needed shuttle students to the football game.

- Let x be the number of buses we have and
- let y be the number of cars that we have.

Suppose that there are at most $M_1 = 5$ buses that could be rented and at most $M_2 = 20$ cars that could be available.

This constraint can be modeled as

$$\begin{aligned} x - 2 &\leq 5\delta \\ y - 10 &\leq 20(1 - \delta) \\ \delta &\in \{0, 1\}, \end{aligned} \tag{1.18}$$

1.7.3. If then implications - opposite direction

Suppose that we want to model the fact that if we have at most 10 students attending this course, then we must switch to a smaller classroom.

Let $x_i \in \{0, 1\}$ be 1 if student i is in the course or not. Let $\delta \in \{0, 1\}$ be 1 if we need to switch to a smaller classroom.

Thus, we want to model

If

$$\sum_{i \in I} x_i \leq 10$$

then

$$\delta = 1.$$

We can model this as

$$\sum_{i \in I} x_i \geq 10 + 1 + M\delta. \tag{1.19}$$

If inequality, then indicator:

We let m be a lower bound on the quantity $a^\top x - b$ and we let ε be a tiny number that is an error bound in verifying if an inequality is violated. **If the data a, b are integer and x is an integer, then we can take $\varepsilon = 1$.**

Now

$$\text{If } a^\top x \leq b \text{ then } \delta = 1 \tag{1.20}$$

can be modeled as

$$a^\top x - b \geq \varepsilon(1 - \delta) + m\delta. \tag{1.21}$$

Proof. We not justify the statement above.

A simple way to understand this constraint is to consider the *contrapositive* of the if then statement that we want to model. The contrapositive says that

$$\text{If } \delta = 0, \text{ then } a^\top x - b > 0. \tag{1.22}$$

Implication	Constraint
If $\delta = 0$, then $a^\top x \leq b$	$a^\top x \leq b + M\delta$
If $a^\top x \leq b$, then $\delta = 1$	$a^\top x \geq m\delta + \varepsilon(1 - \delta)$

Short list: If/then models with a constraint and a binary variable. Here M and m are upper and lower bounds on $a^\top x - b$ and ε is a small number such that if $a^\top x > b$, then $a^\top x \geq b + \varepsilon$.

To show the contrapositive, we set $\delta = 0$. Then the inequality becomes

$$a^\top x - b \geq \varepsilon(1 - 0) + m0 = \varepsilon > 0.$$

Thus, the contrapositive holds.

If instead we wanted a direct proof:

Case 1: Suppose $a^\top x \leq b$. Then $0 \geq a^\top x - b$, which implies that

$$\delta(a^\top x - b) \geq a^\top x - b$$

Therefore

$$\delta(a^\top x - b) \geq \varepsilon(1 - \delta) + m\delta$$

After rearranging

$$\delta(a^\top x - b - m) \geq \varepsilon(1 - \delta)$$

Since $a^\top x - b - m \geq 0$ and $\varepsilon > 0$, the only feasible choice is $\delta = 1$.

Case 2: Suppose $a^\top x > b$. Then $a^\top x - b \geq \varepsilon$. Since $a^\top x - b \geq m$, both choices $\delta = 0$ and $\delta = 1$ are feasible.

By the choice of ε , we know that $a^\top x - b > 0$ implies that $a^\top x - b \geq \varepsilon$.

Since we don't like strict inequalities, we write the strict inequality as $a^\top x - b \geq \varepsilon$ where ε is a small positive number that is a smallest difference between $a^\top x - b$ and 0 that we would typically observe. As mentioned above, if a, b, x are all integer, then we can use $\varepsilon = 1$.

Now we want an inequality with left hand side $a^\top x - b \geq$ and right hand side to take the value

- ε if $\delta = 0$,
- m if $\delta = 1$.

This is accomplished with right hand side $\varepsilon(1 - \delta) + m\delta$. 

Many other combinations of if then statements are summarized in the following table: These two implications can be used to derive the following longer list of implications.

Lastly, if you insist on having exact correspondance, that is, " $\delta = 0$ if and only if $a^\top x \leq b$ " you can simply include both constraints for "if $\delta = 0$, then $a^\top x \leq b$ " and if " $a^\top x \leq b$, then $\delta = 0$ ". Although many problems may be phrased in a way that suggests you need "if and only if", it is often not necessary to use both constraints due to the objectives in the problem that naturally prevent one of these from happening.

Implication	Constraint
If $\delta = 0$, then $a^\top x \leq b$	$a^\top x \leq b + M\delta$
If $\delta = 0$, then $a^\top x \geq b$	$a^\top x \geq b + m\delta$
If $\delta = 1$, then $a^\top x \leq b$	$a^\top x \leq b + M(1 - \delta)$
If $\delta = 1$, then $a^\top x \geq b$	$a^\top x \geq b + m(1 - \delta)$
If $a^\top x \leq b$, then $\delta = 1$	$a^\top x \geq b + m\delta + \varepsilon(1 - \delta)$
If $a^\top x \geq b$, then $\delta = 1$	$a^\top x \leq b + M\delta - \varepsilon(1 - \delta)$
If $a^\top x \leq b$, then $\delta = 0$	$a^\top x \geq b + m(1 - \delta) + \varepsilon\delta$
If $a^\top x \geq b$, then $\delta = 0$	$a^\top x \leq b + M(1 - \delta) - \varepsilon\delta$

Long list: If/then models with a constraint and a binary variable. Here M and m are upper and lower bounds on $a^\top x - b$ and ε is a small number such that if $a^\top x > b$, then $a^\top x \geq b + \varepsilon$.

For example, if we want to add a binary variable δ that means

$$\begin{cases} \delta = 0 \text{ implies } a^\top x \leq b \\ \delta = 1 \text{ Otherwise} \end{cases}$$

If $\delta = 1$ does not effect the rest of the optimization problem, then adding the constraint regarding $\delta = 1$ is not necessary. Hence, typically, in this scenario, we only need to add the constraint $a^\top x \leq b + M\delta$.

1.7.4. Multi Term Disjunction with application to 2D packing

1.8 Generalized k -of- n Disjunction

A disjunction is a generalization of an “or” statement. Suppose that we have n constraints

$$\mathbf{a}_1^\top \mathbf{x} \leq b_1, \quad \mathbf{a}_2^\top \mathbf{x} \leq b_2, \quad \dots, \quad \mathbf{a}_n^\top \mathbf{x} \leq b_n$$

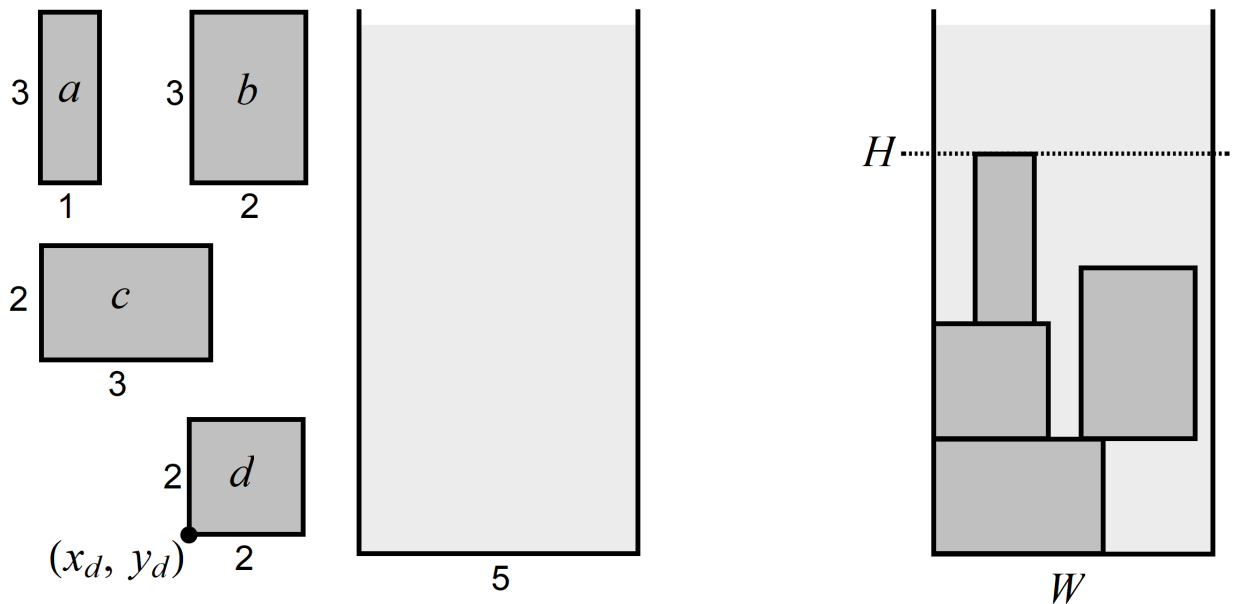
and we want to enforce at least k of them. This can be accomplished linearly by introducing a new binary indicator variable δ_i for each of the disjunctive constraints $i \in \{1, \dots, n\}$:

$$\begin{aligned} \mathbf{a}_1^\top \mathbf{x} &\leq b_1 + M(1 - \delta_1) \\ \mathbf{a}_2^\top \mathbf{x} &\leq b_2 + M(1 - \delta_2) \\ &\vdots \\ \mathbf{a}_n^\top \mathbf{x} &\leq b_n + M(1 - \delta_n) \\ \sum_{i=1}^n \delta_i &\geq k \end{aligned}$$

If $\delta_i = 1$, then the i^{th} disjunctive constraint is actively enforced. The last constraint ($\sum_{i=1}^n \delta_i \geq k$) ensures that at least k of the constraints is active.

1.8.0.1. Strip Packing Problem

Suppose we have a selection of rectangles I and a 2-dimensional strip with width W and infinite height. Each rectangle $i \in I$ has width w_i and height h_i and we want to pack the rectangles into the strip so that (1.1a) overall height is minimized, (1.1b) overall width is less than W , and (1.1c) none of the rectangles overlap. See Figure 1.8 for an example.



(a) Pack the rectangles into the strip

(b) Minimize overall height H .

An Example of the Strip Packing Problem

Let (x_i, y_i) denote the position of the lower-left-hand corner of each rectangle $i \in I$. The overlapping constraint (1.1c) is the trickiest part. Consider a pair of rectangles i and j : rectangle j is located entirely to the left of rectangle i if x_j has a value larger than $x_i + w_i$. That is, if $x_j \geq x_i + w_i$. On the other hand, if $y_j \geq y_i + h_i$, then rectangle j is located entirely above rectangle i . If either of these constraints is satisfied then rectangles i and j do not overlap. We could also place i above or to the right of j . This gives four constraints that we need to satisfy at least one of. The model can be thought of thusly:

$$\text{Minimize } \max_{i \in I} \{y_i + h_i\} \quad (1.1a)$$

$$\text{s.t. } \max_{i \in I} \{x_i + w_i\} \leq W \quad (1.1b)$$

$$\left. \begin{array}{l} x_i + w_i \leq x_j \\ x_j + w_j \leq x_i \\ y_i + h_i \leq y_j \\ y_j + h_j \leq y_i \end{array} \right\} \begin{array}{l} \text{At least one of these} \\ \text{for every distinct pairs} \\ \text{of rectangles } i, j \in I \end{array} \quad (1.1c)$$

$$x_i, y_i \geq 0 \quad \forall i \in I \quad (1.1d)$$

Constraint (1.1c) is a set of four *disjunctive* constraints. This can be expressed linearly by introducing a new binary indicator variable δ_{ijk} for each of the disjunctive constraints in (1.1c):

$$\begin{array}{ll} \text{Minimize} & H \\ \text{s.t.} & y_i + h_i \leq H \quad \forall i \in I \end{array} \quad (1.1a)$$

$$x_i + w_i \leq W \quad \forall i \in I \quad (1.1b)$$

$$\begin{array}{ll} x_i + w_i \leq x_j + M(1 - \delta_{ij1}) & \forall i, j \in I: i > j \\ x_j + w_j \leq x_i + M(1 - \delta_{ij2}) & \forall i, j \in I: i > j \\ y_i + h_i \leq y_j + M(1 - \delta_{ij3}) & \forall i, j \in I: i > j \\ y_j + h_j \leq y_i + M(1 - \delta_{ij4}) & \forall i, j \in I: i > j \end{array} \quad (1.1c)$$

$$\sum_{k=1}^4 \delta_{ijk} \geq 1 \quad \forall i, j \in I: i > j$$

$$x_i, y_i \geq 0 \quad \forall i \in I$$

$$\delta_{ij} \in \{0, 1\}^4 \quad \forall i, j \in I: i > j$$

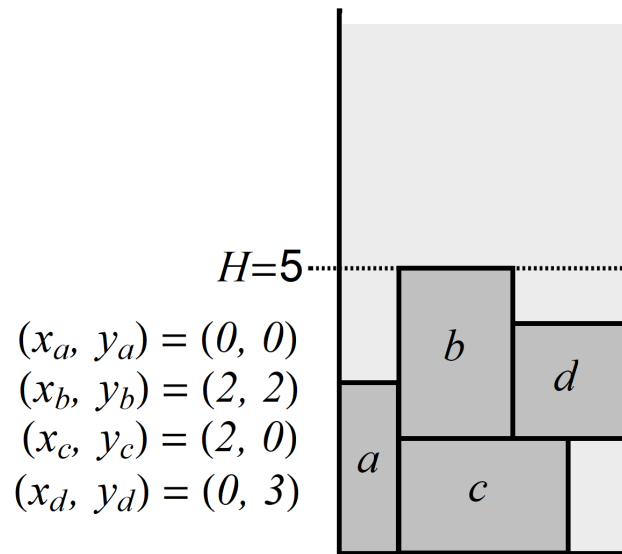
If $\delta_{ijk} = 1$, then the k^{th} disjunctive constraint is actively enforced. The last constraint ($\sum_{k=1}^4 \delta_{ijk} \geq 1$) ensures that at least one of the constraints is active. An optimal solution to the example is given in Figure 1.9; it was found via GurobiPy.

This problem is considered strongly NP-Hard.

1.8.1. SOS1 Constraints

Definition 1.1. Special Ordered Sets of Type 1 (SOS1) A Special Ordered Sets of type 1 (SOS1) constraint on a vector indicates that *at most one element of the vector can non-zero*.

We next give an example of how to use binary variables to model this and then show how much simpler it can be coded using the SOS1 constraint.



The Optimal Solution to the example problem

Example 1.24: SOS1 Constraints

Gurobipy

Solve the following optimization problem:

$$\begin{aligned}
 &\text{maximize} && 3x_1 + 4x_2 + x_3 + 5x_4 \\
 &\text{subject to} && 0 \leq x_i \leq 5 \\
 &&& \text{at most one of the } x_i \text{ can be nonzero}
 \end{aligned}$$

1.8.2. SOS2 Constraints

Definition 1.2. Special Ordered Sets of Type 2 (SOS2) A *Special Ordered Set of Type 2 (SOS2)* constraint on a vector indicates that *at most two elements of the vector can non-zero AND the non-zero elements must appear consecutively*.

We next give an example of how to use binary variables to model this and then show how much simpler it can be coded using the SOS2 constraint.

Example 1.25: SOS2

Gurobipy

Solve the following optimization problem:

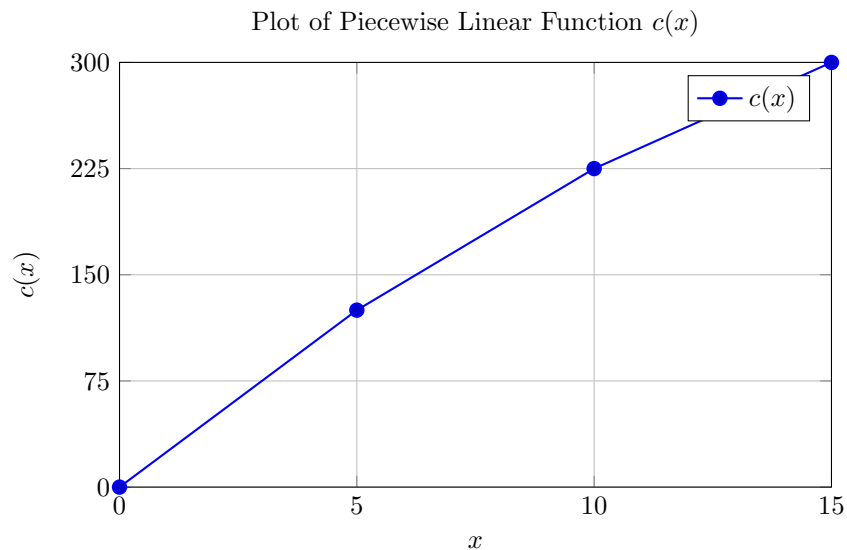
$$\begin{aligned}
 &\text{maximize} && 3x_1 + 4x_2 + x_3 + 5x_4 \\
 &\text{subject to} && 0 \leq x_i \leq 5 \\
 &&& \text{at most two of the } x_i \text{ can be nonzero} \\
 &&& \text{and the nonzero } x_i \text{ must be consecutive}
 \end{aligned}$$

1.8.3. Piecewise linear functions with SOS2 constraint**Example 1.26: Piecewise Linear Function**

Gurobipy

Consider the piecewise linear function $c(x)$ given by

$$c(x) = \begin{cases} 25x & \text{if } 0 \leq x \leq 5 \\ 20x + 25 & \text{if } 5 \leq x \leq 10 \\ 15x + 75 & \text{if } 10 \leq x \leq 15 \end{cases}$$



We will use integer programming to describe this function. We will fix $x = a$ and then the integer program

will set the value y to $c(a)$.

$$\begin{aligned}
 \min \quad & 0 \\
 \text{s.t.} \quad & x = 5z_2 + 10z_3 + 15z_4 \\
 & y = 125z_2 + 225z_3 + 300z_4 \\
 & z_1 + z_2 + z_3 + z_4 = 1 \\
 & \text{SOS2} : \{z_1, z_2, z_3, z_4\} \\
 & 0 \leq z_i \leq 1 \quad \text{for all } i \in \{1, 2, 3, 4\} \\
 & x = a
 \end{aligned}$$

Example 1.27: Piecewise Linear Function Application

code:pwl-application

Consider the following optimization problem where the objective function includes the term $c(x)$, where $c(x)$ is the piecewise linear function described in Example 1.26:

$$\max \quad z = 12x_{11} + 12x_{21} + 14x_{12} + 14x_{22} - c(x) \quad (1.2)$$

$$\text{s.t.} \quad x_{11} + x_{12} \leq x + 5 \quad (1.3)$$

$$x_{21} + x_{22} \leq 10 \quad (1.4)$$

$$0.5x_{11} - 0.5x_{21} \geq 0 \quad (1.5)$$

$$0.4x_{12} - 0.6x_{22} \geq 0 \quad (1.6)$$

$$x_{ij} \geq 0 \quad (1.7)$$

$$0 \leq x \leq 15 \quad (1.8)$$

Given the piecewise linear, we can model the whole problem explicitly as a mixed-integer linear program.

$$\begin{aligned}
 \max \quad & 12X_{1,1} + 12X_{2,1} + 14X_{1,2} + 14X_{2,2} - y \\
 \text{s.t.} \quad & x - 5z_2 - 10z_3 - 15z_4 = 0 \\
 & y - 125z_2 - 225z_3 - 300z_4 = 0 \\
 & z_1 + z_2 + z_3 + z_4 = 1 \\
 & X_{1,1} + X_{1,2} - x \leq 5 \\
 & X_{2,1} + X_{2,2} \leq 10 \\
 & 0.5X_{1,1} - 0.5X_{2,1} \geq 0 \\
 & 0.4X_{1,2} - 0.6X_{2,2} \geq 0 \\
 & \text{SOS2} : \{z_1, z_2, z_3, z_4\} \\
 & X_{i,j} \geq 0 \quad \forall i \in \{1, 2\}, j \in \{1, 2\} \\
 & 0 \leq z_i \leq 1 \quad \forall i \in \{1, 2, 3, 4\} \\
 & 0 \leq x \leq 15 \\
 & y \text{ free}
 \end{aligned} \quad (1.9)$$

1.8.3.1. SOS2 with binary variables

Modeling Piecewise linear function

- Write down pairs of breakpoints and functions values $(a_i, f(a_i))$.
- Define a binary variable z_i indicating if x is in the interval $[a_i, a_{i+1}]$.
- Define multipliers λ_i such that x is a combination of the a_i 's and therefore the output $y = f(x)$ is a combination of the $f(a_i)$'s.
- Restrict that at most 2 λ_i 's are non-zero and that those 2 are consecutive.

$$\begin{aligned}
 \min \quad & \sum_{i=1}^k \lambda_i f(a_i) \\
 \text{s.t.} \quad & \sum_{i=1}^k \lambda_i = 1 \\
 & x = \sum_{i=1}^k \lambda_i a_i \\
 & \lambda_1 \leq z_1 \\
 & \lambda_i \leq z_{i-1} + z_i \quad \text{for all } i = 2, \dots, k-1, \\
 & \lambda_k \leq z_{k-1} \\
 & \lambda_i \geq 0, z_i \in \{0, 1\}.
 \end{aligned}$$

1.8.4. Maximizing a minimum

When the constraints could be general, we will write $x \in X$ to define general constraints. For instance, we could have $X = \{x \in \mathbb{R}^n : Ax \leq b\}$ or $X = \{x \in \mathbb{R}^n : Ax \leq b, x \in \mathbb{Z}^n\}$ or many other possibilities.

Consider the problem

$$\begin{aligned}
 \max \quad & \min\{x_1, \dots, x_n\} \\
 \text{such that} \quad & x \in X
 \end{aligned}$$

Having the minimum on the inside is inconvenient. To remove this, we just define a new variable y and enforce that $y \leq x_i$ and then we maximize y . Since we are maximizing y , it will take the value of the smallest x_i . Thus, we can recast the problem as

$$\begin{array}{ll}\max & y \\ \text{such that} & y \leq x_i \text{ for } i = 1, \dots, n \\ & x \in X\end{array}$$

1.8.5. Relaxing (nonlinear) equality constraints

There are a number of scenarios where the constraints can be relaxed without sacrificing optimal solutions to your problem. In a similar vein of the maximizing a minimum, if because of the objective we know that certain constraints will be tight at optimal solutions, we can relax the equality to an inequality. For example,

$$\begin{array}{ll}\max & x_1 + x_2 + \dots + x_n \\ \text{such that} & x_i = y_i^2 + z_i^2 \text{ for } i = 1, \dots, n\end{array}$$

1.8.6. Connecting to continuous variables

$$1. \text{ If } x_i > 0, \text{ then } y_i = 1. \tag{1.10}$$

$$2. \text{ If } x_i = 0, \text{ then } y_i = 0. \tag{1.11}$$

$$3. \text{ If } y_i = 1, \text{ then } x_i \geq 5 \tag{1.12}$$

1.8.7. Modeling the Exact Absolute Value

In many optimization models, we need to enforce an exact equality of the form $|x| = t$, where x is a decision variable and $t \geq 0$. Unlike the relaxation $|x| \leq t$ (which is convex and can be handled with two linear inequalities), the exact equality $|x| = t$ describes a *non-convex* set and cannot be represented using linear constraints alone.

To model this, we decompose x into its positive and negative parts: let $x^+, x^- \geq 0$ with

$$x = x^+ - x^-, \quad t = x^+ + x^-.$$

This decomposition correctly gives $|x| = t$ provided that x^+ and x^- are not both strictly positive simultaneously. We enforce this complementarity condition using a binary variable $z \in \{0, 1\}$ and a large constant

M (an a priori upper bound on $|x|$):

$$\begin{aligned} x &= x^+ - x^-, \\ t &= x^+ + x^-, \\ x^+ &\leq Mz, \\ x^- &\leq M(1-z), \\ x^+, x^- &\geq 0, \quad z \in \{0, 1\}. \end{aligned}$$

When $z = 1$, the constraint $x^- \leq 0$ forces $x^- = 0$, so $x = x^+ \geq 0$. When $z = 0$, the constraint $x^+ \leq 0$ forces $x^+ = 0$, so $x = -x^- \leq 0$. In either case, exactly one of x^+ and x^- is active, and $t = |x|$ holds exactly.

1.8.7.1. Modeling the Exact 1-Norm

Building on the absolute value formulation, we can model the exact ℓ_1 -norm equality

$$\sum_{i=1}^n |x_i| = c,$$

where $\mathbf{x} \in \mathbb{R}^n$ is a vector of decision variables and $c > 0$ is a constant. Constraints of this type arise, for instance, in portfolio optimization models that require full investment (where short-selling is permitted).

We introduce positive and negative parts for each component: $x_i = x_i^+ - x_i^-$ with $x_i^+, x_i^- \geq 0$. Binary variables $z_i \in \{0, 1\}$ enforce that at most one of x_i^+ and x_i^- is nonzero:

$$\begin{aligned} x_i &= x_i^+ - x_i^-, \quad i = 1, \dots, n, \\ x_i^+ &\leq cz_i, \quad i = 1, \dots, n, \\ x_i^- &\leq c(1 - z_i), \quad i = 1, \dots, n, \\ \sum_{i=1}^n (x_i^+ + x_i^-) &= c, \\ x_i^+, x_i^- &\geq 0, \quad z_i \in \{0, 1\}, \quad i = 1, \dots, n. \end{aligned}$$

Here we use c as the big- M bound, since no individual component can exceed c in absolute value when the total 1-norm equals c .

1.8.7.2. Modeling the Exact Maximum

Suppose we wish to model the exact equality $t = \max\{x_1, \dots, x_n\}$, where $x_1, \dots, x_n$ and t are decision variables. The relaxed version $t \geq x_i$ for all i only gives $t \geq \max_i x_i$; to enforce equality, we need to ensure that t actually equals one of the x_i .

We introduce binary indicator variables $z_1, \dots, z_n$, where $z_i = 1$ means that x_i achieves the maximum. Choosing a sufficiently large constant M , the formulation is:

$$\begin{aligned} x_i &\leq t, \quad i = 1, \dots, n, \\ t &\leq x_i + M(1 - z_i), \quad i = 1, \dots, n, \\ z_1 + z_2 + \dots + z_n &= 1, \\ z_i &\in \{0, 1\}, \quad i = 1, \dots, n. \end{aligned}$$

The first set of constraints ensures $t \geq x_i$ for all i . The second set, combined with exactly one $z_i = 1$, ensures that for the index i with $z_i = 1$, we have $t \leq x_i$, which together with $t \geq x_i$ gives $t = x_i$. For all other indices $j \neq i$, the big- M term relaxes the upper bound constraint.

For further reading on these and many related mixed-integer modeling patterns, see the MOSEK Modeling Cookbook⁷, which surveys conic and integer reformulations of nonsmooth functions.

⁷MOSEK ApS, *MOSEK Modeling Cookbook*, <https://docs.mosek.com/modeling-cookbook/>.

1.9 Job Shop Scheduling

Try it out visually!

Job-Shop Schedule (Gantt): this chapter's scheduling problem, solved and animated.

The Job Shop Scheduling Problem (JSSP) is a classical combinatorial optimization problem that is NP-hard. A set of jobs must be processed on a set of machines, where each job follows a prescribed sequence of operations across the machines. Each operation has a fixed processing time, each machine can handle only one job at a time, and once a machine begins processing an operation it must finish without interruption. The objective is to minimize the *makespan*, the time at which the last job completes.

Consider a small workshop with three machines (m_1, m_2, m_3) and four jobs. Each job must visit each machine exactly once, in a job-specific order, with the following processing times (in hours):

- Job j_1 : $m_1(3) \rightarrow m_2(2) \rightarrow m_3(4)$.
- Job j_2 : $m_2(2) \rightarrow m_1(4) \rightarrow m_3(1)$.
- Job j_3 : $m_3(3) \rightarrow m_1(1) \rightarrow m_2(3)$.
- Job j_4 : $m_1(2) \rightarrow m_3(3) \rightarrow m_2(2)$.

Processing jobs one after another (sequentially) yields a makespan of $9 + 7 + 7 + 7 = 30$ hours. By allowing jobs to run concurrently on different machines, the makespan can be significantly reduced.

1.9.1. JSSP Components

Two pieces of data specify an instance. First, we collect the n jobs in a set $J = \{1, \dots, n\}$ and the m machines in a set $I = \{1, \dots, m\}$. Second, each job carries its own route through the shop together with the time it needs at each stop: we write o_r^j for the machine on which job j runs its r -th operation, so that the route of job j is the sequence $O^j = (o_1^j, \dots, o_m^j)$, and we write $p_{ij} \geq 0$ for the time job j occupies machine i .

A *schedule* assigns a start time to every operation. Three rules separate the feasible schedules from the infeasible ones: the operations of a job may not start out of order—each waits for its predecessor on the route O^j to finish; a machine hosts at most one job at any moment; and processing cannot be paused, so an operation that starts at time t on machine i blocks that machine until time $t + p_{ij}$. Among all feasible schedules we want one whose makespan—the time when the final operation ends—is as small as possible.

Example 1.5. Job Shop Scheduling Problem Input Data

Basic Parameters. Consider $n = 4$ jobs and $m = 3$ machines, using the workshop data introduced above.

Processing Times. The matrix of processing times (one row per job, one column per machine) is:

$$P = \begin{bmatrix} 3 & 2 & 4 \\ 4 & 2 & 1 \\ 1 & 3 & 3 \\ 2 & 2 & 3 \end{bmatrix}$$

The entry in row j , column i gives the processing time p_{ij} of job j on machine i .

Machine Sequences. The order in which each job visits the machines is:

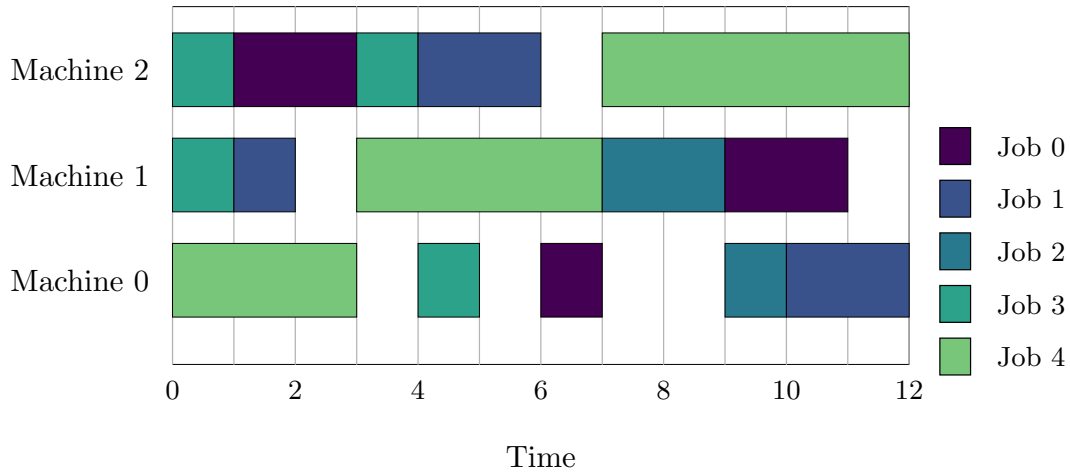
$$O = \begin{bmatrix} 1 & 2 & 3 \\ 2 & 1 & 3 \\ 3 & 1 & 2 \\ 1 & 3 & 2 \end{bmatrix}$$

Each row gives the machine sequence for a job. For instance, job 1 is processed first on machine 1, then machine 2, then machine 3, while job 3 starts on machine 3, then visits machine 1, and finishes on machine 2.

Big M Calculation. A large constant M is used in the big- M formulation to enforce disjunctive constraints. A valid choice is the sum of all processing times:

$$M = \sum_{i=1}^m \sum_{j=1}^n p_{ij} = (3 + 2 + 4) + (4 + 2 + 1) + (1 + 3 + 3) + (2 + 2 + 3) = 30.$$

Gantt Chart of JSSP Solution



Job shop scheduling solution shown as a Gantt chart.

APPLICATIONS OF JSSP The JSSP has numerous practical applications, including:

- Manufacturing: Scheduling tasks in a production line to optimize throughput.
- Computer Science: Allocating tasks to processors in parallel computing environments.
- Healthcare: Scheduling surgeries and other treatments in hospitals.
- Transportation: Optimizing maintenance tasks for vehicles or aircraft in a maintenance facility.

1.9.2. Mathematical Model

We now build a mixed-integer program from the ingredients above. This *disjunctive* formulation is classical and is due to Manne⁸.

Sets:

- $J = \{1, \dots, n\}$, the jobs.
- $I = \{1, \dots, m\}$, the machines.

Parameters:

- o_r^j , the machine hosting the r -th operation of job j ; row j of the matrix O from the example lists these machines in order.
- p_{ij} , the processing time of job j on machine i .
- M , a big- M constant; the total processing time $\sum_{i \in I} \sum_{j \in J} p_{ij}$ is always a valid choice, as computed for the example above.

Variables: The main decisions are continuous start times

$$s_{ij} = \text{start time of job } j \text{ on machine } i,$$

together with a variable $C_{\max}$ for the makespan. Start times alone cannot express “one of these two jobs must wait for the other,” so for each machine i and each pair of jobs $j < k$ we add a binary variable that records which job goes first:

$$z_{ijk} = \begin{cases} 1, & \text{when machine } i \text{ processes job } j \text{ before job } k, \\ 0, & \text{when job } k \text{ takes its turn on machine } i \text{ first,} \end{cases} \quad i \in I, j, k \in J, j < k.$$

Constraints: *Precedence within a job.* Each job must follow its route: operation r of job j cannot begin until operation $r - 1$ has finished, that is,

$$s_{o_r^j} \geq s_{o_{r-1}^j} + p_{o_{r-1}^j} \quad \forall j \in J, r \in \{2, \dots, m\}.$$

One job at a time per machine. Fix a machine i and two jobs $j < k$. Feasibility demands that either job j clears the machine before job k arrives, $s_{ij} + p_{ij} \leq s_{ik}$, or the reverse, $s_{ik} + p_{ik} \leq s_{ij}$. This is precisely

⁸A. S. Manne, “On the Job-Shop Scheduling Problem,” *Operations Research* 8(2):219–223, 1960.

the either-or situation of Section ??, and applying that recipe with z_{ijk} in the role of the selector variable δ yields the big- M pair

$$\begin{aligned} s_{ij} + p_{ij} &\leq s_{ik} + M(1 - z_{ijk}) & \forall i \in I, j, k \in J, j < k, \\ s_{ik} + p_{ik} &\leq s_{ij} + Mz_{ijk} & \forall i \in I, j, k \in J, j < k. \end{aligned}$$

When $z_{ijk} = 1$ the first inequality forces job j ahead of job k while the second is switched off; when $z_{ijk} = 0$ the roles swap.

Makespan. The makespan must sit above the completion time of every job's last operation:

$$C_{\max} \geq s_{o_m^j} + p_{o_m^j} \quad \forall j \in J.$$

Model:

$$\begin{aligned} \min \quad & C_{\max} \\ \text{s.t.} \quad & s_{o_r^j} \geq s_{o_{r-1}^j} + p_{o_{r-1}^j} & \forall j \in J, r \in \{2, \dots, m\} & \text{follow the route of job } j \\ & s_{ij} + p_{ij} \leq s_{ik} + M(1 - z_{ijk}) & \forall i \in I, j, k \in J, j < k & j \text{ first when } z_{ijk} = 1 \\ & s_{ik} + p_{ik} \leq s_{ij} + Mz_{ijk} & \forall i \in I, j, k \in J, j < k & k \text{ first when } z_{ijk} = 0 \\ & C_{\max} \geq s_{o_m^j} + p_{o_m^j} & \forall j \in J & \text{makespan covers every job} \\ & s_{ij} \geq 0 & \forall i \in I, j \in J \\ & z_{ijk} \in \{0, 1\} & \forall i \in I, j, k \in J, j < k \\ & C_{\max} \geq 0 \end{aligned} \tag{1.1a}$$

Solving this model on the four-job, three-machine instance above gives an optimal makespan of 12 hours—a substantial improvement over the 30 hours needed to run the jobs one at a time.

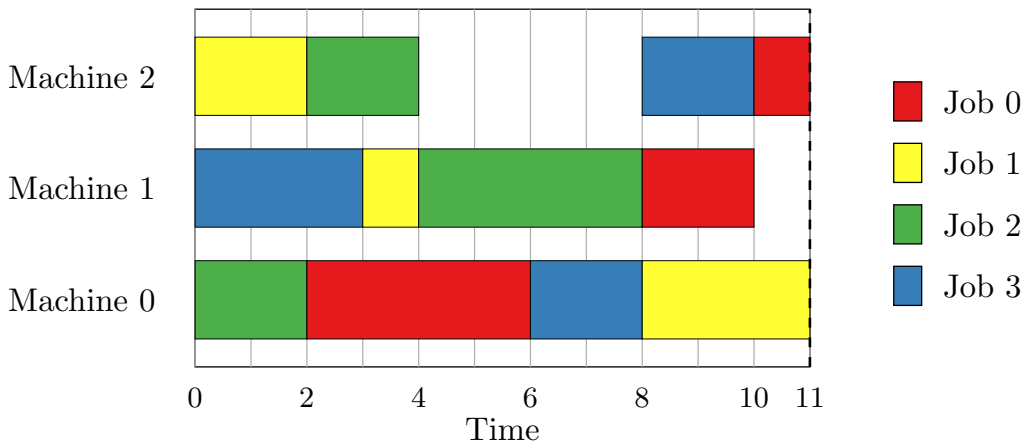
Example 1.28: Duplo Scheduling

Excel PuLP Gurobipy

The Duplo in-class exercise is solved with Gurobi in the link.

The optimal value is 11.

Gantt Chart of JSSP Solution



Gantt chart of the Duplo job shop scheduling solution with three machines and four color-coded jobs.



Photograph of Duplo building blocks arranged as a physical demonstration of the job shop schedule.

1.9.3. Job Shop Scheduling Variations

Makespan minimization of assigning jobs to machines.

In this variation, each machine can handle a number of different types of jobs. Some machines can do certain jobs faster than others.

We want to minimize the completion time of all of the jobs.

In this variation, each job visits its listed machines in sequence: most jobs require processing on two machines, in the order given, while a few need only one.

Machines:

- Machine 1: Can perform jobs 1, 2, 3, 4, and 5
- Machine 2: Can perform jobs 6, 7, 8, 9, and 10
- Machine 3: Can perform jobs 11, 12, 13, 14, and 15
- Machine 4: Can perform jobs 1, 6, 11, 2, and 7
- Machine 5: Can perform jobs 3, 8, 13, 4, and 9

Jobs:

- Job 1: Machine 1 (processing time = 2 hours), Machine 4 (processing time = 1 hour)
- Job 2: Machine 1 (processing time = 3 hours), Machine 4 (processing time = 2 hours)
- Job 3: Machine 1 (processing time = 2 hours), Machine 5 (processing time = 1 hour)
- Job 4: Machine 1 (processing time = 4 hours), Machine 5 (processing time = 2 hours)
- Job 5: Machine 1 (processing time = 1 hour)
- Job 6: Machine 2 (processing time = 3 hours), Machine 4 (processing time = 2 hours)
- Job 7: Machine 2 (processing time = 2 hours), Machine 4 (processing time = 1 hour)
- Job 8: Machine 2 (processing time = 4 hours), Machine 5 (processing time = 2 hours)
- Job 9: Machine 2 (processing time = 1 hour), Machine 5 (processing time = 1 hour)

- Job 10: Machine 2 (processing time = 2 hours)
- Job 11: Machine 3 (processing time = 3 hours), Machine 4 (processing time = 2 hours)
- Job 12: Machine 3 (processing time = 2 hours), Machine 4 (processing time = 1 hour)
- Job 13: Machine 3 (processing time = 4 hours), Machine 5 (processing time = 2 hours)
- Job 14: Machine 3 (processing time = 1 hour), Machine 5 (processing time = 1 hour)
- Job 15: Machine 3 (processing time = 2 hours)

Objective: Minimize the makespan (i.e. the total time it takes to complete all the jobs)

Constraints:

- Each job can only be performed on the specified machines in the order listed
- A machine can only work on one job at a time
- There are no precedence constraints between jobs.

We leave this as an exercise for the reader to model and solve.

1.10 Quadratic Assignment Problem (QAP)

Resources

- *An applied case of quadratic assignment problem in hospital department layout*
- *See Quadratic Assignment Problem: A survey and Applications.*
- *Quadratic Assignment Problem in Airline Gate Assignment to minimize walking distance*

The quadratic assignment problem must choose the assignment of n facilities to n locations. Each facility sends some flow to each other facility, and there is a distance to consider between locations. The objective is to minimize to distance times the flow of the assignment.

Example: Hospital Layout On any given day in the hospital, there will be patients that move from various locations in the hospital to various other locations in the hospital. For example, patients move from the operating room to a recovery room, or from the emergency room to the operating room, etc.

We would like to chose the locations of these places in the hospital to minimize the amount of total distance traveled by all the patients.

Quadratic Assignment Problem:

NP-Complete

Given flow f_{ij} connections c_{ij} and fixed building costs f_i , demands d_j and capacities u_i , the capacitated facility location problem is

Sets:

- Let $I = \{1, \dots, n\}$ be the set of facilities.
- Let $K = \{1, \dots, n\}$ be the set of locations.

Parameters:

- f_{ij} - flow from facility i to facility j .
- d_{kl} - distance from location k to location l .
- c_{ik} - cost to setup facility i at location k .

Variables:

- Let

$$x_{ik} = \begin{cases} 1 & \text{if we place facility } i \text{ in location } k, \\ 0 & \text{otherwise.} \end{cases}$$

Model:

$$\begin{aligned} \min \quad & \sum_{i=1}^n \sum_{j=1}^n \sum_{k=1}^n \sum_{l=1}^n f_{ij} d_{kl} x_{ik} x_{jl} + \sum_{i=1}^n \sum_{k=1}^n c_{ik} x_{ik} && \text{(total cost)} \\ \text{s.t.} \quad & \sum_{i=1}^n x_{ik} = 1 && \text{for all } k = 1, \dots, n \text{ (assign facility to location } k) \\ & \sum_{k=1}^n x_{ik} = 1 && \text{for all } i = 1, \dots, n \\ & && \text{(assign one location to facility } i) \\ & x_{ik} \in \{0, 1\} && \text{for all } i = 1, \dots, n, \text{ and } k = 1, \dots, n \\ & && \text{(binary decisions)} \end{aligned}$$

1.11 Generalized Assignment Problem (GAP)

This section is adapted from the Wikipedia article *Generalized assignment problem*⁹. In applied mathematics, the maximum **generalized assignment problem** is a problem in combinatorial optimization.

⁹Wikipedia contributors, "Generalized assignment problem," https://en.wikipedia.org/wiki/Generalized_assignment_problem, licensed CC BY-SA; text edited and reformatted for this book.

This problem is a generalization of the assignment problem in which both tasks and agents have a size. Moreover, the size of each task might vary from one agent to the other.

This problem in its most general form is as follows: There are a number of agents and a number of tasks. Any agent can be assigned to perform any task, incurring some cost and profit that may vary depending on the agent-task assignment. Moreover, each agent has a budget and the sum of the costs of tasks assigned to it cannot exceed this budget. It is required to find an assignment in which all agents do not exceed their budget and total profit of the assignment is maximized.

1.11.1. In special cases

In the special case in which all the agents' budgets and all tasks' costs are equal to 1, this problem reduces to the assignment problem. When the costs and profits of all tasks do not vary between different agents, this problem reduces to the multiple knapsack problem. If there is a single agent, then, this problem reduces to the knapsack problem.

1.11.2. Explanation of definition

In the following, we have n kinds of items, a_1 through a_n and m kinds of bins b_1 through b_m . Each bin b_i is associated with a budget t_i . For a bin b_i , each item a_j has a profit p_{ij} and a weight w_{ij} . A solution is an assignment from items to bins. A feasible solution is a solution in which for each bin b_i the total weight of assigned items is at most t_i . The solution's profit is the sum of profits for each item-bin assignment. The goal is to find a maximum profit feasible solution.

Mathematically the generalized assignment problem can be formulated as an integer program:

$$\text{maximize } \sum_{i=1}^m \sum_{j=1}^n p_{ij} x_{ij}. \quad (1.1)$$

$$\text{subject to } \sum_{j=1}^n w_{ij} x_{ij} \leq t_i \quad i = 1, \dots, m; \quad (1.2)$$

$$\sum_{i=1}^m x_{ij} = 1 \quad j = 1, \dots, n; \quad (1.3)$$

$$x_{ij} \in \{0, 1\} \quad i = 1, \dots, m, \quad j = 1, \dots, n; \quad (1.4)$$

1.12 Other material

1.12.1. Binary reformulation of integer variables

If an integer variable has small upper and lower bounds, it can sometimes be advantageous to recast it as a sequence of binary variables - for either modeling, the solver, or both. Although there are technically

many ways to do this, here are the two most common ways.

Full reformulation:

u many binary variables

For a non-negative integer variable x with upper bound u , modeled as

$$0 \leq x \leq u, \quad x \in \mathbb{Z}, \quad (1.1)$$

this can be reformulated with u binary variables $z_1, \dots, z_u$ as

$$\begin{aligned} x &= \sum_{i=1}^u i z_i = z_1 + 2z_2 + \dots + u z_u \\ 1 &\geq \sum_{i=1}^u z_i = z_1 + z_2 + \dots + z_u \\ z_i &\in \{0, 1\} \quad \text{for } i = 1, \dots, u \end{aligned} \quad (1.2)$$

We call this the *full reformulation* because there is a binary variable z_i associated with every value i that x could take. That is, if $z_3 = 1$, then the second constraint forces $z_i = 0$ for all $i \neq 3$ (that is, z_3 is the only non-zero binary variable), and hence by the first constraint, $x = 3$.

Binary reformulation:

$O(\log u)$ many binary variables

For a non-negative integer variable x with upper bound u , modeled as

$$0 \leq x \leq u, \quad x \in \mathbb{Z}, \quad (1.3)$$

this can be reformulated with u binary variables $z_1, \dots, z_{\lfloor \log(u) \rfloor}$ as

$$\begin{aligned} x &= \sum_{i=0}^{\lfloor \log(u) \rfloor} 2^i z_i = z_0 + 2z_1 + 4z_2 + 8z_3 + \dots + 2^{\lfloor \log(u) \rfloor} z_{\lfloor \log(u) \rfloor} \\ z_i &\in \{0, 1\} \quad \text{for } i = 1, \dots, \lfloor \log(u) \rfloor \end{aligned} \quad (1.4)$$

We call this the *log reformulation* because this requires only logarithmically many binary variables in terms of the upper bound u . This reformulation is particularly better than the full reformulation when the upper bound u is a “larger” number, although we will leave it ambiguous as to how larger a number need to be in order to be described as a “larger” number.

1.13 Exercises

Exercise 1.1 (Locating Distribution Terminals)

Suppose that a company based in St. John's is considering adding distribution terminals in (1) Halifax, (2) Moncton, (3) Montréal, (4) Ottawa, and (5) Toronto.

The cost of building the five terminals in millions of dollars would be 10 in Halifax, 12 in Moncton, 20 in Montréal, 18 in Ottawa, and 25 in Toronto.

- 1. No more than two terminals may be built.*
- 2. If no terminal is built in Halifax, then one must be built in Moncton.*
- 3. At least one terminal must be built in central Canada (i.e. at locations (3), (4), or (5)).*

Define the variables: $y_i = \begin{cases} 1 & \text{if a terminal is built in city } i (i = 1, \dots, 5) \\ 0 & \text{otherwise} \end{cases}$

Using these variables, write a (binary) integer program that models the problem of determining which terminals to build subject to the constraints above that minimizes cost.

1.14 Literature and Resources

Resources

- *The AIMMS modeling has many great examples. It can be book found here: AIMMS Modeling Book.*
- *MIT Open Courseware*
- *For many real world examples, see this book Case Studies in Operations Research Applications of Optimal Decision Making, edited by Murty, Katta G. Or find it here.*
- *GUROBI modeling examples by GUROBI*
- *GUROBI modeling examples by Open Optimization that are linked in this book*

Knapsack Problem

- *Video! - Michel Belaire (EPFL) teaching knapsack problem*

Set Cover

- *Video! - Michel Belaire (EPFL) explaining set covering problem*
- *See AIMMS - Media Selection for an example of set covering applied to media selection.*

Facility Location

- *Wikipedia - Facility Location Problem*
- *See GUROBI Modeling Examples - Facility Location.*

Other examples

- *Sudoku*
- *AIMMS - Employee Training*
- *AIMMS - Media Selection*
- *AIMMS - Diet Problem*
- *AIMMS - Farm Planning Problem*
- *AIMMS - Pooling Problem*
- *INFORMS - Impact*
- *INFORMS - Success Story - Bus Routing*

Notes from AIMMS modeling book.

- *AIMMS - Practical guidelines for solving difficult MILPs*
- *AIMMS - Linear Programming Tricks*
- *AIMMS - Formulating Optimization Models*
- *AIMMS - Practical guidelines for solving difficult linear programs*

Modeling Tricks

- *JuMP tips and tricks*
- *Mosek Modeling Cookbook*

Further Topics

- *Precedence Constraints*